

Beginner AWS Architecture and DevOps Guide

Proper architecture diagram, DevOps pipeline and concise AWS labs for absolute beginners

Flipkart Clone Microservices Workspace

Beginner AWS Architecture and DevOps Pipeline Guide

This is the beginner-friendly guide for understanding the **architecture**, the **DevOps pipeline**, and the **AWS deployment steps** for the Flipkart Clone workspace.

Use this if you are an absolute beginner and want the simple path.

Project path:

```
/home/user/flipkart-clone
```

Final ZIP:

```
/home/user/flipkart-clone.zip
```

1. What Are We Deploying?

You are deploying a Flipkart-style e-commerce platform with these services:

Service	Purpose	Port
Frontend	React UI served by Nginx	80 in AWS, 8080 locally
API Gateway	Public API entry point	3000
User Service	Register, login, profile	3001
Product Service	Products, categories, search	3002
Cart Service	Shopping cart	3003
Order Service	Orders and checkout	3004
Payment Service	Payment simulation	3005
Notification Service	In-app notifications	3006
DocumentDB	MongoDB-compatible database	27017

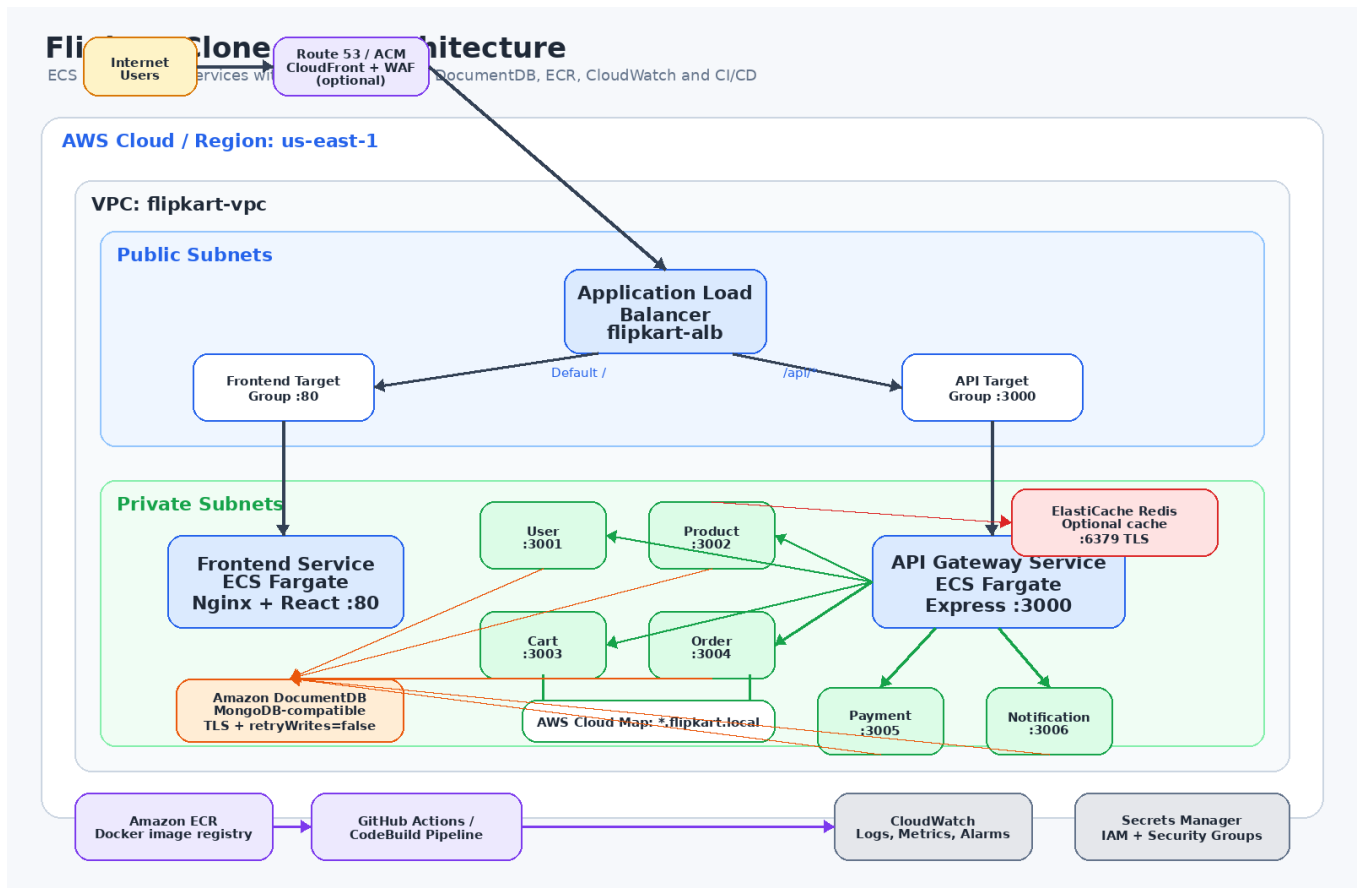
Public API route style:

```
/api/users/login  
/api/users/register  
/api/products  
/api/cart/:userId  
/api/orders  
/api/payments  
/api/notifications/user/:userId
```

Important:

```
Use /api/products  
Do not use /api/products/products
```

2. Proper AWS Architecture Diagram



Simple Explanation

1. Users open the website in browser.
2. Browser reaches Application Load Balancer.
3. ALB sends normal website traffic / to Frontend service.
4. ALB sends API traffic /api/* to API Gateway.
5. API Gateway forwards requests to internal microservices.
6. Microservices communicate using Cloud Map names like `user-service.flipkart.local`.
7. Services store data in Amazon DocumentDB.
8. Docker images are stored in ECR.
9. ECS Fargate runs containers.
10. CloudWatch collects logs, metrics and alarms.

3. Best DevOps Pipeline Diagram

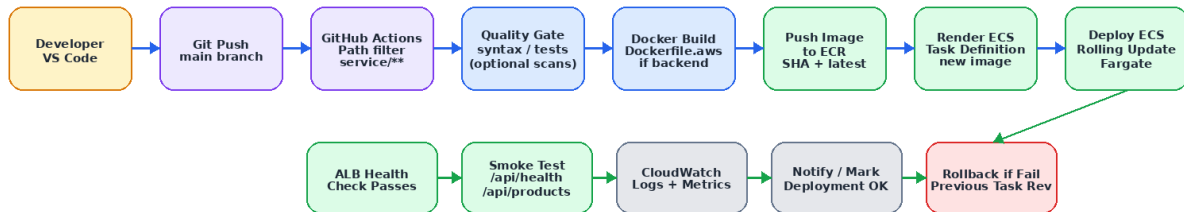
Recommended DevOps Pipeline

Best flow: validate infrastructure first, then automate per-service zero-downtime deployments to ECS

Phase 1: Infrastructure Validation Gate — do this before CI/CD



Phase 2: Automated CI/CD Pipeline



Manual workflow: deploy-to-aws-ecs.yml
Run all services or selected service from GitHub Actions

Automatic workflows: deploy-<service>.yml
Path-based deploy only changed service for safer releases

Why This Pipeline Is Best for This Project

This project has multiple microservices. The best pipeline is:

Per-service automatic deployment + manual all-service deployment

That means:

- If you change only product-service, only product service deploys.
- If you change only frontend, only frontend deploys.
- If you want to deploy all, use the manual workflow.

This is safer, faster and cheaper than rebuilding every service every time.

4. Recommended Pipeline Stages

Automatic Per-Service Pipeline

Each service has a workflow:

```

.github/workflows/deploy-api-gateway.yml
.github/workflows/deploy-user-service.yml
.github/workflows/deploy-product-service.yml
.github/workflows/deploy-cart-service.yml
.github/workflows/deploy-order-service.yml
.github/workflows/deploy-payment-service.yml
.github/workflows/deploy-notification-service.yml
.github/workflows/deploy-frontend.yml
  
```

Stages:

Stage	What It Does
1. Trigger	Runs when code changes in one service folder
2. Checkout	Downloads code from GitHub
3. AWS Auth	Uses GitHub secrets to connect to AWS
4. ECR Login	Logs in to Amazon ECR
5. Docker Build	Builds service image, using <code>Dockerfile.aws</code> for backend services
6. Tag Image	Tags image with Git SHA and <code>latest</code>
7. Push Image	Pushes Docker image to ECR
8. Download Task Definition	Gets current ECS task definition
9. Render New Task Definition	Updates image URI in task definition
10. Deploy to ECS	Starts rolling deployment
11. Wait for Stability	Waits until ECS service is stable
12. Verify	You test ALB/API and check CloudWatch logs

Manual All-Service Pipeline

File:

```
.github/workflows/deploy-to-aws-ecs.yml
```

Use this from GitHub Actions when you want to deploy:

```
all services
or one selected service manually
```

5. Beginner AWS Deployment Roadmap

Follow this exact order.

```
Step 0: Run locally
Step 1: Create AWS account and CLI profile
Step 2: Create ECR repositories
Step 3: Push Docker images
Step 4: Create VPC and security groups
Step 5: Create DocumentDB
Step 6: Create ALB and target groups
Step 7: Create ECS cluster
Step 8: Create Cloud Map namespace
Step 9: Create task definitions
Step 10: Create ECS services
Step 11: Validate infrastructure
Step 12: Enable CI/CD
Step 13: Monitor with CloudWatch
```

Do not start CI/CD until the manual infrastructure deployment works.

6. Step 0: Run Locally First

```
cd /home/user/flipkart-clone
cp .env.example .env
docker compose up --build -d
docker compose ps
```

```
curl http://localhost:3000/health
```

Seed products:

```
docker compose exec product-service npm run seed
```

Open:

```
http://localhost:8080
```

If this does not work locally, do not go to AWS yet.

7. Step 1: AWS Account and CLI

1. Create AWS account at <https://aws.amazon.com/>.
2. Select region: us-east-1.
3. Create IAM user or use IAM Identity Center.
4. Install AWS CLI.
5. Configure:

```
aws configure
```

Use:

```
Region: us-east-1  
Output: json
```

Verify:

```
aws sts get-caller-identity
```

8. Step 2: Create ECR Repositories

You need 8 repositories:

```
flipkart-api-gateway  
flipkart-user-service  
flipkart-product-service  
flipkart-cart-service  
flipkart-order-service  
flipkart-payment-service  
flipkart-notification-service  
flipkart-frontend
```

Fast way:

```
cd /home/user/flipkart-clone  
./scripts/aws-create-ecr-repositories.sh
```

Console way:

```
AWS Console → ECR → Create repository → Private
```

Enable scan on push.

9. Step 3: Push Docker Images to ECR

Get account ID:

```
aws sts get-caller-identity --query Account --output text
```

Push all images:

```
cd /home/user/flipkart-clone
export AWS_ACCOUNT_ID=123456789012
export AWS_REGION=us-east-1
./push-to-ecr.sh
```

This uses `Dockerfile.aws` for backend services when available.

10. Step 4: Create VPC and Security Groups

Use AWS Console:

VPC → Create VPC → VPC and more

Recommended:

Setting	Value
Name	flipkart-vpc
CIDR	10.0.0.0/16
AZs	2
Public subnets	2
Private subnets	2
NAT Gateway	1 per AZ for production, none for cheap dev

Security groups:

SG	Inbound
flipkart-alb-sg	80, 443 from internet
flipkart-api-gateway-sg	3000 from ALB SG
flipkart-services-sg	3001-3006 from API Gateway SG and itself
flipkart-docdb-sg	27017 from services SG
flipkart-redis-sg	6379 from services SG, optional

11. Step 5: Create DocumentDB

Console:

DocumentDB → Create cluster

Use:

Cluster name: flipkart-docdb-cluster
Username: docdbadmin
Private subnets
Security group: flipkart-docdb-sg

Use this URI pattern in ECS:

```
mongodb://docdbadmin:URL_ENCODED_PASSWORD@DOCDB_ENDPOINT:27017/flipkart-products?tls=true&tlsCAFile=/app/global-bundle.pem&replicaSet=rs0&readPreference=secondaryPreferred&retryWrites=false
```

Change DB name per service.

12. Step 6: Create ALB and Target Groups

Create target groups:

```
flipkart-frontend-tg → port 80 → health /healthz  
flipkart-api-tg      → port 3000 → health /health
```

Create ALB:

Name: flipkart-alb
Scheme: Internet-facing
Subnets: public subnets
Security group: flipkart-alb-sg
Default action: frontend target group

Listener rule:

```
/api/* → flipkart-api-tg
```

13. Step 7: Create ECS Cluster

Console:

ECS → Clusters → Create cluster

Use:

Name: flipkart-cluster
Infrastructure: Fargate
Container Insights: Enabled

14. Step 8: Create Cloud Map Namespace

Console:

Cloud Map → Create namespace

Use:

```
Namespace: flipkart.local
Type: Private DNS namespace
VPC: flipkart-vpc
```

Service names should become:

```
user-service.flipkart.local
product-service.flipkart.local
cart-service.flipkart.local
order-service.flipkart.local
payment-service.flipkart.local
notification-service.flipkart.local
```

15. Step 9: Create ECS Task Definitions

Templates are included:

```
devops/ecs-task-definitions/
```

Create task definitions for:

```
flipkart-api-gateway
flipkart-user-service
flipkart-product-service
flipkart-cart-service
flipkart-order-service
flipkart-payment-service
flipkart-notification-service
flipkart-frontend
```

Replace placeholders:

```
<ACCOUNT_ID>
<AWS_REGION>
<DOCDB_CLUSTER_ENDPOINT>
<URL_ENCODED_PASSWORD>
<JWT_SECRET>
```

16. Step 10: Create ECS Services

Create internal services first:

```
user-service
product-service
cart-service
order-service
payment-service
notification-service
```

Settings:

```
Launch type: Fargate
Subnets: private subnets
Security group: flipkart-services-sg
Public IP: disabled
Service discovery: enabled
Load balancer: none
```

Create API Gateway service:

```
Service name: api-gateway-service
Security group: flipkart-api-gateway-sg
Target group: flipkart-api-tg
```

Create frontend service:

```
Service name: frontend-service
Target group: flipkart-frontend-tg
```

17. Step 11: Validate Infrastructure Before CI/CD

Use the final validation document:

```
docs/PHASE_1_INFRA_VALIDATION_PHASE_2_CICD.pdf
```

Or run helper:

```
cd /home/user/flipkart-clone
cp scripts/aws-env.example scripts/aws-env.local
# edit ALB_DNS_NAME, AWS account values
./scripts/aws-validate-deployment-template.sh
```

Do not enable CI/CD until these pass:

```
ECS services running
ALB targets healthy
Frontend loads
/api/products works
Register/login works
Cart works
Checkout creates order
CloudWatch logs have no critical errors
```

18. Step 12: Enable GitHub CI/CD

Push project to GitHub.

Add secrets:

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
AWS_REGION=us-east-1
AWS_ACCOUNT_ID
ECS_CLUSTER=flipkart-cluster
REACT_APP_API_URL=/api
```

Automatic workflows are already included:

```
.github/workflows/deploy-api-gateway.yml
.github/workflows/deploy-user-service.yml
.github/workflows/deploy-product-service.yml
.github/workflows/deploy-cart-service.yml
.github/workflows/deploy-order-service.yml
.github/workflows/deploy-payment-service.yml
.github/workflows/deploy-notification-service.yml
.github/workflows/deploy-frontend.yml
```

Manual all-service workflow:

`.github/workflows/deploy-to-aws-ecs.yml`

19. Step 13: Monitoring

Minimum monitoring:

CloudWatch logs for every service
 ECS CPU and memory dashboard
 ALB 5XX alarm
 ALB unhealthy target alarm
 DocumentDB CPU alarm
 AWS Budget alert

Use:

`docs/MONITORING_AND_OPERATIONS_GUIDE.pdf`

20. Beginner Troubleshooting

Problem	Check First
Website not opening	ALB DNS, frontend target group health
API gives 502	API Gateway target group, API logs
Login fails	User service logs, DocumentDB URI
Products empty	Product DB seed data
Cart fails	Cart service can reach Product service
Order fails	Order service can reach Cart and Notification services
ECS task stops	CloudWatch task logs
Image pull error	ECR repo name and ECS task execution role
Mongo timeout	DocumentDB SG, TLS URI, Dockerfile.aws

21. Final Beginner Checklist

- ☐ Local Docker app works
- ☐ ECR repos created
- ☐ Images pushed to ECR
- ☐ VPC created
- ☐ Security groups created
- ☐ DocumentDB available
- ☐ ALB created
- ☐ ECS cluster created
- ☐ Cloud Map created
- ☐ Task definitions created
- ☐ ECS services running
- ☐ ALB targets healthy
- ☐ Browser app works
- ☐ `/api/products` works
- ☐ Register/login/cart/order tested
- ☐ CloudWatch logs checked
- ☐ GitHub secrets added
- ☐ CI/CD manual run tested
- ☐ CI/CD auto run tested
- ☐ Monitoring alarms created

When all are checked, your project is AWS-ready and DevOps-ready.